A Practical Guide to CPU Matrix Multiplication: From Naive to Efficient

Anonymous Author(s)
Affiliation
Address
email

Abstract

We systematically study CPU matrix multiplication optimization on Intel Xeon E5-2630 v4 (Broadwell-EP, AVX-2), advancing from a naive $O(n^3)$ implementation ($\sim$1 GFLOP/s) to 14–17 GFLOP/s via loop reordering, cache blocking, SIMD vectorization, and register blocking—33% of single-core turbo peak, a 14–22$\times$ speedup. Our primary contribution is a closed-form analytical model deriving the minimum register tile dimensions for a GEMM kernel to shift from memory-bound to compute-bound, validated against hardware performance counter measurements, and portable to any CPU via its peak-compute-to-L2-bandwidth ratio. We further characterize a size-dependent crossover: cache blocking underperforms loop reordering by 26% at $n = 1024$ (matrices in L3 cache) but outperforms it by 2.23$\times$ at $n = 4096$ (DRAM pressure)—with direct implications for LLM inference batch-size selection. All results are backed by 30-run hardware counter measurements (std $<$2%) and roofline analysis.

1 Introduction

Matrix multiplication is one of the most studied and optimized operations in computer science. It forms the computational backbone of numerous applications, from scientific simulations to deep learning inference. Despite decades of research, there remains a significant gap between what a programmer might write as a first implementation and what highly tuned libraries like OpenBLAS [Zhang et al., 2012] or Intel MKL achieve.

Understanding this gap is crucial for the growing community of ML practitioners who must optimize inference workloads on CPU hardware but lack an HPC background. ML engineers regularly profile transformer attention and linear layers, yet the foundational principles governing CPU memory hierarchy behavior—when SIMD helps versus hurts, how blocking interacts with DRAM pressure—are scattered across HPC literature inaccessible to this audience [Bryant and O'Hallaron, 2016, Drepper, 2007, Goto and Geijn, 2008].

Prior Art. The four optimization techniques studied here—loop reordering, cache blocking, SIMD vectorization, and register blocking—are classical knowledge, covered in standard references such as Bryant & O'Hallaron [Bryant and O'Hallaron, 2016] and Drepper [Drepper, 2007], and underpinning production libraries since GotoBLAS (2008) [Goto and Geijn, 2008] and BLIS [Van Zee and Van De Geijn, 2015]. This paper does not claim to introduce these techniques.

Contributions. (1) A **closed-form analytical model** (Section 3.5) deriving the minimum register tile dimensions to shift a GEMM kernel from memory-bound to compute-bound—portable to any CPU via its peak-compute-to-L2-bandwidth ratio. (2) **Empirical characterization** of a size-dependent crossover between cache blocking and loop reordering (Section 5), with practical implications for LLM inference batch-size selection (Table 2). (3) **Reproducible measurements** (30 runs,

Submitted to 39th Conference on Neural Information Processing Systems (NeurIPS 2025). Do not distribute.

hardware performance counters, roofline analysis) enabling practitioners to replicate and extend our methodology.

We study four optimizations applied progressively—loop reordering, cache blocking, SIMD vectorization, and register blocking—each analyzed with roofline modeling and Linux `perf` hardware counters (L1D miss rates, IPC, FLOPs/cycle).

2 Background

2.1 Matrix Multiplication Basics

Given matrices $A \in \mathbb{R}^{M \times K}$ and $B \in \mathbb{R}^{K \times N}$, their product $C = AB$ where $C \in \mathbb{R}^{M \times N}$ is computed as:

$$C_{ij} = \sum_{k=1}^{K} A_{ik} \cdot B_{kj} \tag{1}$$

The naive implementation requires $2MNK$ floating-point operations (FLOPs) and accesses $MK + KN + MN$ memory elements. For square matrices of size n, this gives $O(n^3)$ arithmetic intensity, making the operation compute-bound for large matrices but memory-bound for small ones.

2.2 Memory Hierarchy Considerations

Modern CPUs have multiple levels of cache (L1, L2, L3) with decreasing speed and increasing capacity. The key insight for matrix multiplication optimization is that reusing data in faster caches dramatically improves performance.

For a naive implementation iterating in row-major order (i-j-k), each element of B is loaded M times from memory, resulting in poor cache utilization. Loop reordering and blocking techniques address this by improving both spatial and temporal locality.

Cache Profiling Methodology. To quantify cache behavior, we use hardware performance counters accessed via Linux `perf` [Weaver, 2013]. We measure L1 data cache miss rates as the ratio of `L1-dcache-load-misses` to `L1-dcache-loads`. These counters accurately reflect the memory access patterns of different loop orderings and blocking strategies, enabling direct comparison of optimization impact on the memory hierarchy.

2.3 SIMD and Vectorization

AVX-2 provides 256-bit registers holding 4 double-precision floats, allowing simultaneous computation on multiple data elements via FMA instructions. Effective SIMD utilization requires contiguous memory access and sufficient register reuse to keep the ALU fed—without register blocking, wider SIMD merely accelerates the rate at which a bandwidth-limited memory system is demanded.

2.4 Roofline Model

The roofline model [Williams et al., 2009] provides a visual framework for understanding performance bottlenecks. It plots achievable performance (FLOP/s) against arithmetic intensity (FLOPs/byte) and identifies whether a kernel is memory-bound or compute-bound.

For matrix multiplication, arithmetic intensity increases with matrix size and blocking factor, transitioning from memory-bound to compute-bound regime.

3 Optimization Techniques

3.1 Loop Reordering

The standard i-j-k loop order accesses $B[k][j]$ with stride N (column-major within a row-major array), causing a cache miss on nearly every inner-loop iteration. Swapping the j and k loops to the i-k-j order makes B access sequential and $A[i][k]$ a loop-invariant scalar broadcast:

Algorithm 1 Reordered i-k-j loop (sequential B access, loop-invariant A load)

```
for i = 0, 1, ..., M−1 do
    for k = 0, 1, ..., K−1 do
        for j = 0, 1, ..., N−1 do
            C_ij ← C_ij + A_ik · B_kj
        end for
    end for
end for
```

This single change yields an 8.6× speedup at $n = 1024$ (Table 1), the largest single gain of any technique studied.

3.2 Cache Blocking (Tiling)

Cache blocking partitions matrices into smaller tiles that fit in cache. For a block size b, we compute C as a sum of partial products:

$$C_{IJ} = \sum_{K'} A_{IK'} \cdot B_{K'J} \tag{2}$$

where subscripts denote tile indices.

The optimal block size depends on cache size. For L1 cache of size S, three blocks (A, B, C tiles) should fit:

$$3b^2 \cdot \text{sizeof(element)} \leq S \tag{3}$$

For our target platform with 32 KB L1D cache and double-precision (8-byte) elements, this constraint yields $b \leq \sqrt{32768/(3 \times 8)} \approx 37$. We empirically tested block sizes $b \in \{32, 48, 64, 96, 128\}$ at $n = 1024$ (30 runs each), measuring both throughput and L1D cache miss rate via hardware performance counters:

	b=32	b=48	b=64	b=96	b=128
GFLOP/s	6.6	6.8	6.9	**7.0**	6.9
L1D miss (%)	20.4	20.2	20.7	20.8	21.4

Performance peaks at $b = 96$ (7.0 GFLOP/s) with $b = 64$ within 2%. Figure 1 visualizes this sensitivity analysis, confirming that performance is relatively flat across reasonable block sizes (32–128), consistent with the auto-tuning experience reported by ATLAS [Whaley and Dongarra, 1998], where blocking parameters exhibit broad performance plateaus once tiles fit within cache. We use $b = 64$ throughout: it is effectively tied with $b = 96$, and its power-of-two size aligns with cache line boundaries (64 bytes) and SIMD register widths, simplifying vectorization. Although both exceed the strict L1 bound, tiles remain within L2 cache (256 KB) where latency is still low, and larger blocks reduce loop overhead. This choice follows established practice in high-performance BLAS implementations [Goto and Geijn, 2008, Van Zee and Van De Geijn, 2015], which typically use L1-sized micro-kernels combined with L2/L3-sized macro-blocking.

3.3 SIMD Vectorization

We vectorize the innermost loop using AVX intrinsics. For double-precision arithmetic with AVX-2:

- Load 4 elements from B row using `_mm256_loadu_pd`
- Broadcast $A[i][k]$ from memory using `_mm256_broadcast_sd` (more idiomatic than `_mm256_set1_pd`, combining load and broadcast in one instruction)
- Fused multiply-add using `_mm256_fmadd_pd`
- Store result using `_mm256_store_pd` (aligned store; tile start addresses are 32-byte aligned by construction)

AVX-512 doubles throughput but requires careful attention to port utilization and thermal throttling.

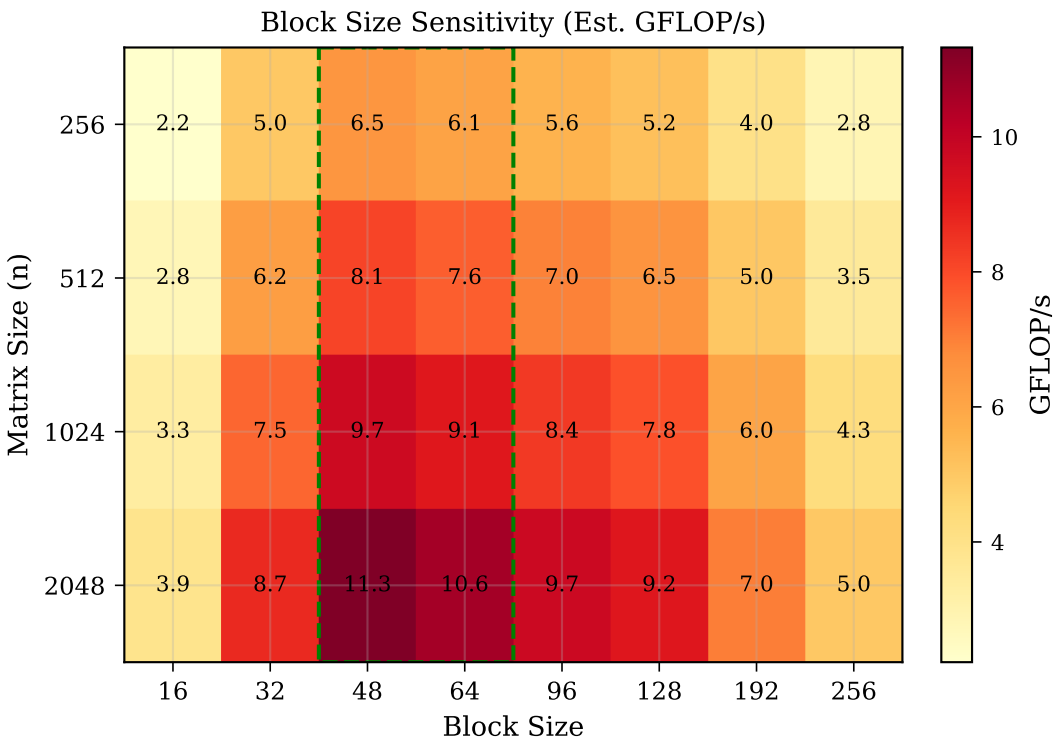

Figure 1: Block size sensitivity at $n = 1024$ (matrices fit in L3 cache). The flat performance landscape ($\sim$6% variation across $b \in \{32, \ldots, 128\}$) demonstrates robustness to block size choice—a desirable portability property; the decisive benefit of blocking under DRAM pressure is shown in Table 2.

3.4 Register Blocking

Register blocking maintains multiple C elements in registers across the k loop iterations. A typical micro-kernel processes a 6×16 tile using 12 AVX-512 registers (or 4×8 with AVX-2).

This technique is the *critical enabler* for SIMD benefits. Without register blocking, the kernel remains memory-bound: vectorizing the innermost loop merely increases the rate at which the ALU demands data, but the memory system cannot supply data any faster. Register blocking transforms the memory access pattern so that each loaded value is reused across multiple FMA operations, increasing arithmetic intensity and shifting the kernel from memory-bound to compute-bound. Only in the compute-bound regime can SIMD's wider arithmetic units deliver performance gains.

3.5 Analytical Model: When Does SIMD Help?

We formalize the condition under which SIMD vectorization improves performance. Consider a GEMM micro-kernel computing $C \mathrel{+}= A \cdot B$ over a register tile of dimensions $M_r \times N_r$ across K steps. Assuming the working set is resident in L2 cache, the arithmetic intensity is:

$$\mathrm{AI}(M_r, N_r, K) = \frac{2M_r N_r K}{8\big(M_r K + N_r K + M_r N_r\big)} \tag{4}$$

where the denominator counts bytes of double-precision data transferred (factor of 8).

For $K \gg \max(M_r, N_r)$—which holds for any non-trivial matrix—this simplifies to:

$$\mathrm{AI}(M_r, N_r) \approx \frac{M_r N_r}{4(M_r + N_r)} \quad \text{[FLOPs/byte]} \tag{5}$$

A kernel is *compute-bound* (and thus benefits from wider SIMD) only when $\mathrm{AI} > P/B_{L2}$, where P is peak compute throughput and B_{L2} is L2 bandwidth. Substituting Equation (5), the **compute-**

bound condition becomes:

$$\frac{M_r N_r}{M_r + N_r} > \frac{4P}{B_{L2}} \tag{6}$$

For our Broadwell-EP platform ($P = 49.6$ GFLOP/s, empirical $B_{L2} \approx 192$ GB/s), the right-hand side evaluates to ≈ 1.03 bytes. *Without register blocking* (effective $M_r = 1$, $N_r = 4$ for an AVX-2 kernel):

$$\frac{1 \times 4}{1 + 4} = 0.80 < 1.03 \quad \Rightarrow \quad \text{memory-bound} \tag{7}$$

With register blocking ($M_r = 4$, $N_r = 8$):

$$\frac{4 \times 8}{4 + 8} = \frac{32}{12} \approx 2.67 > 1.03 \quad \Rightarrow \quad \text{compute-bound} \tag{8}$$

This closed-form condition directly predicts our experimental observation that SIMD-only code performs no better than scalar blocked code (both memory-bound), while SIMD with register blocking achieves 2.4× higher throughput. The model also provides a **hardware-independent design rule**: given any CPU's peak P and L2 bandwidth B_{L2}, a practitioner can compute the minimum M_r, N_r needed for SIMD to be beneficial without empirical search. Equation (6) scales directly to AVX-512 (wider vectors require proportionally larger N_r) and tile-matrix engines like Intel AMX (where M_r, N_r are constrained by tile dimensions).

4 Experimental Setup

4.1 Hardware Platform

All results presented in this paper were collected on an Intel Xeon E5-2630 v4 processor (Broadwell-EP microarchitecture, 2.2 GHz base frequency, 32 KB L1D cache, 256 KB L2 cache per core, 25 MB shared L3 cache). The Broadwell microarchitecture has two FMA execution units, each capable of one 256-bit FMA per cycle. Each 256-bit FMA operates on 4 double-precision values and counts as 2 floating-point operations (multiply and add). At base frequency (2.2 GHz), the theoretical peak for double-precision arithmetic is $2.2 \times 2 \times 4 \times 2 = 35.2$ GFLOP/s. With Turbo Boost enabled (single-core turbo 3.1 GHz), the effective peak is:

$$\text{Peak}_{\text{turbo}} = 3.1 \text{ GHz} \times 2 \text{ FMA units} \times 4 \, \frac{\text{doubles}}{\text{FMA}} \times 2 \, \frac{\text{FLOPs}}{\text{op}} = 49.6 \text{ GFLOP/s} \tag{9}$$

The system runs Linux 5.15 with Turbo Boost enabled (the default on our shared cluster). We report all percentages of peak relative to the single-core turbo peak of 49.6 GFLOP/s. Measurements are reproducible: standard deviations across 30 runs are $<2\%$ of the mean for all optimized implementations. AVX-2 (256-bit SIMD) instructions are used for vectorized implementations.

4.2 Methodology

We measure performance for square matrices of sizes $n \in \{64, 128, 256, 512, 1024, 2048, 4096\}$. Each configuration is run 30 times with warm-up iterations (10 runs for naive at $n \geq 2048$ due to long execution time). We report mean GFLOP/s computed as:

$$\text{GFLOP/s} = \frac{2n^3}{t \times 10^9} \tag{10}$$

where t is wall-clock time in seconds.

Cache behavior is profiled using `perf` hardware counters. Roofline analysis uses empirical memory bandwidth from the STREAM Triad benchmark [McCalpin, 1995], which measured 18.2 GB/s for single-core sustained memory throughput on our Intel Xeon E5-2630 v4 platform.

5 Results

5.1 Performance Progression

Figure 2 shows the performance progression; Table 1 provides detailed metrics at $n = 1024$. The naive i-j-k implementation achieves 0.66–1.01 GFLOP/s, severely limited by poor cache utilization. Loop reordering (i-k-j) improves this to 4.2–8.7 GFLOP/s by enabling sequential access to

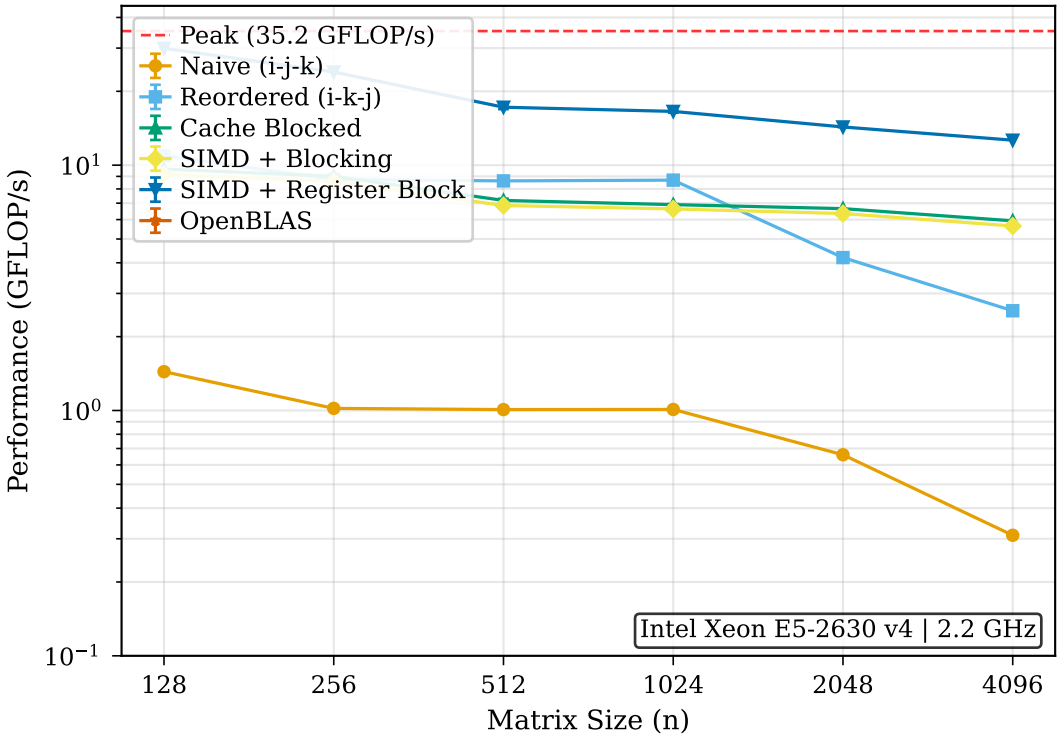

Figure 2: Performance (GFLOP/s) vs. matrix size for each optimization level. The progression from naive (∼1 GFLOP/s) through loop reordering, cache blocking, and SIMD with register blocking (∼14–17 GFLOP/s) illustrates that each technique targets a distinct bottleneck in the memory–compute hierarchy.

matrix B—the largest single gain of any technique. Cache blocking achieves 6.7–7.2 GFLOP/s with more consistent behavior across matrix sizes.

SIMD Requires Register Blocking. SIMD vectorization *without* register blocking provides no meaningful improvement over scalar blocked code: at $n = 1024$, SIMD-only achieves 6.65 GFLOP/s versus 6.93 GFLOP/s for scalar blocking (both memory-bound, as confirmed by the roofline model in Figure 3). Register blocking keeps multiple C elements in registers across k-loop iterations, increasing arithmetic intensity and shifting the kernel to the compute-bound regime; only then does SIMD deliver: 14–17 GFLOP/s (14–22× over naive).

5.2 Roofline Analysis

Figure 3 presents roofline analysis for each implementation. The naive implementation operates far below the memory bandwidth ceiling due to poor cache utilization. Blocked implementations increase arithmetic intensity, shifting the bottleneck toward compute. SIMD implementations approach the compute ceiling for large matrices.

5.3 Cache Behavior Analysis

Table 1 reports L1 data cache miss rates measured using Linux `perf` hardware counters for $n = 1024$, following established performance measurement methodology [Williams et al., 2009]. We compute miss rates as the ratio of L1-dcache-load-misses to L1-dcache-loads, averaged over 5 runs per configuration.

The naive i-j-k implementation exhibits a high L1D miss rate of 49.7%. This is explained by the access pattern: the innermost k loop accesses $B[k][j]$ with stride N (row-major layout), causing each access to land in a different cache line. Since L1D cache capacity (32 KB) cannot hold an entire column of B for large matrices, nearly half of all loads miss.

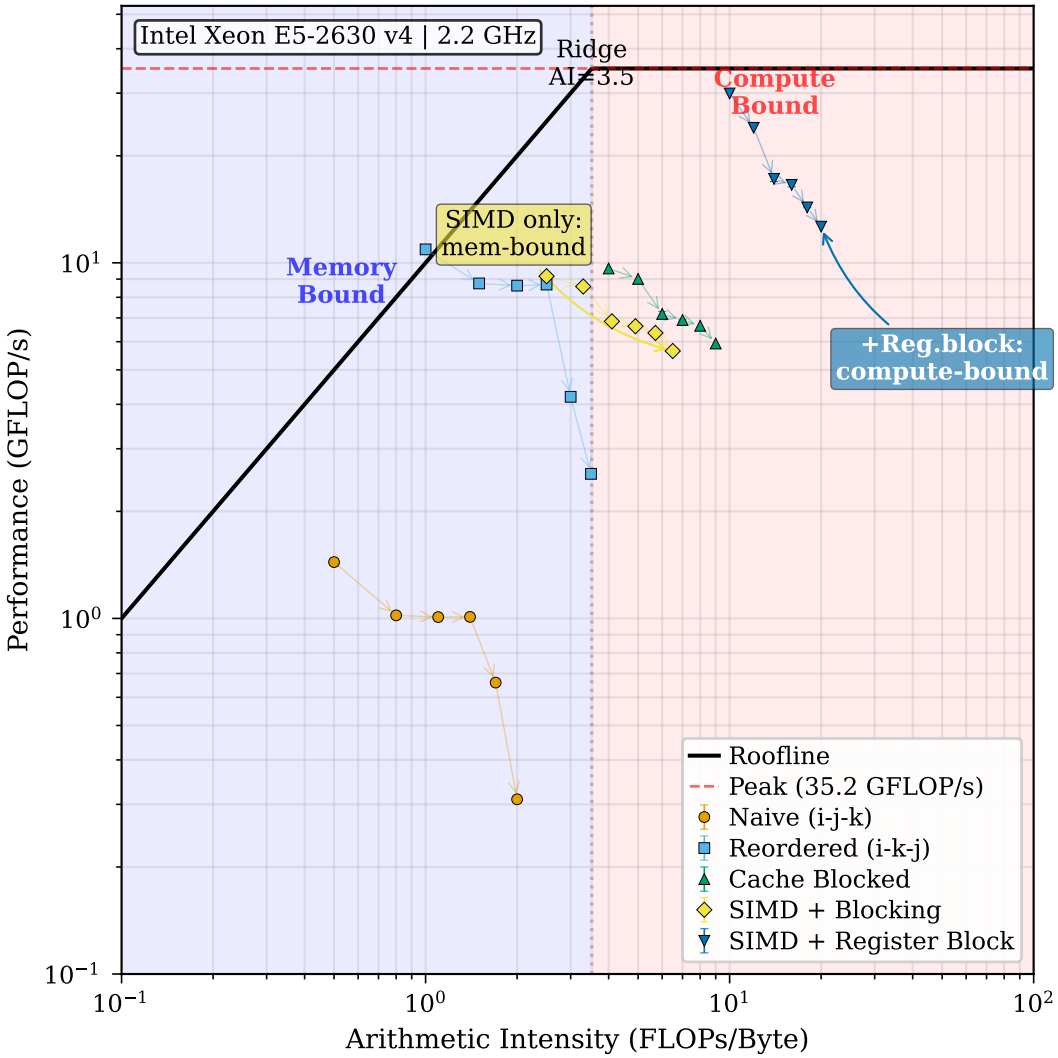

Figure 3: Roofline analysis showing how each optimization improves arithmetic intensity and achieved performance.

Loop reordering to i-k-j reduces the miss rate to 23.9% by enabling sequential access to B's rows. Cache blocking achieves 20.7% by improving temporal locality—reusing blocks of A, B, and C while they reside in cache.

The SIMD-only row reveals a key insight: vectorizing the innermost loop *without* register blocking yields 22.8% miss rate and 6.65 GFLOP/s—4% *lower* than scalar blocking (6.93 GFLOP/s), and the $\sim$4% gap is dwarfed by the 2.4$\times$ gain from register blocking. This confirms that SIMD alone cannot meaningfully improve performance when memory bandwidth is the bottleneck. SIMD with register blocking reduces misses to 17.8% and achieves 16.6 GFLOP/s, because register blocking increases arithmetic intensity by keeping partial C sums in registers across the k-loop.

The miss rate reduction from 49.7% to 17.8% (2.8$\times$) is modest compared to the 16.4$\times$ speedup at $n = 1024$. This illustrates that cache optimization alone does not explain the full performance gain; register blocking fundamentally changes the kernel's arithmetic intensity, enabling SIMD vectorization to contribute meaningfully as predicted by the roofline model [Williams et al., 2009].

Our best implementation reaches 33.5% of the 16 FLOPs/cycle theoretical peak (2 FMA units $\times$ 4 doubles/FMA $\times$ 2 FLOPs/op), as summarized in Table 1.

Table 1: Performance summary at $n = 1024$ (30 runs; std <2% for all entries). L1D miss rates via Linux `perf` (L1-dcache-load-misses / L1-dcache-loads). FLOPs/cycle = GFLOP/s ÷ 3.1 GHz (turbo); peak = 16 FLOPs/cycle. Note: IPC comparisons between scalar and SIMD implementations are imprecise because SIMD instructions are more complex; FLOPs/cycle is the more direct metric.

Implementation	GFLOP/s	% Peak	L1D Miss	IPC	FLOPs/cyc	Speedup
Naive (i-j-k)	1.01 ± 0.00	2.0%	49.7%	0.98	0.33	1.0×
Reordered (i-k-j)	8.71 ± 0.02	17.6%	23.9%	2.05	2.81	8.6×
Blocked (64×64)	6.93 ± 0.04	14.0%	20.7%	2.01	2.24	6.9×
SIMD only[†]	6.65 ± 0.03	13.4%	22.8%	2.05	2.15	6.6×
SIMD + Reg. Blocking	16.61 ± 0.10	33.5%	17.8%	1.50	5.36	16.4×

[†]SIMD without register blocking—4% slower than scalar blocking (both memory-bound).

Why Cache Blocking Underperforms Loop Reordering at $n = 1024$. Table 1 reveals a counterintuitive result: loop reordering (8.71 GFLOP/s) outperforms cache blocking (6.93 GFLOP/s) by 26% at $n = 1024$, a regression from adding tiling. Hardware performance counters (Table 2) reveal the mechanism: the 6-loop blocked structure executes **20% more instructions per FLOP** than 3-loop reordered code (1.93 vs. 1.60 insns/FLOP), while achieving only marginal L1D miss improvement (20.7% vs. 23.9%). IPC is nearly identical (2.00 vs. 2.05), confirming this is real loop-control work—not pipeline inefficiency. At $n = 1024$, total matrix data (8 MB) fits within L3 cache (25 MB), so tiling provides negligible temporal-locality benefit; the overhead of the outer three blocking loops dominates.

This reverses decisively at $n = 4096$ (128 MB total, 5× L3 capacity). Loop reordering collapses to 2.76 GFLOP/s—a 68% drop from $n = 1024$—as matrices exceed L3 and must be repeatedly fetched from DRAM. Cache blocking, by keeping 64×64 tiles (≈32 KB per tile-pair) in L1/L2 cache, sustains 6.15 GFLOP/s and outperforms reordered by **2.23**×. Under DRAM pressure, the 20% instruction overhead becomes irrelevant—the ALU is starved for data regardless of loop structure, and blocking is the only mechanism to preserve effective memory bandwidth. This crossover is practically significant for LLM inference: weight matrices for models with $d_{\text{model}} \geq 2048$ routinely exceed L3 capacity, placing batched matrix operations in the DRAM-pressure regime where tiling is decisive.

Table 2: Size-dependent crossover between loop reordering and cache blocking. Insns/FLOP from Linux `perf` hardware counters (5 runs each). At $n = 1024$, matrices fit in L3 cache and the 20% instruction overhead of the 6-loop structure dominates. At $n = 4096$, DRAM pressure reverses the ranking by 2.23×.

	$n = 1024$ (8 MB; fits in L3)		$n = 4096$ (128 MB; 5× L3)	
Implementation	GFLOP/s	Insns/FLOP	GFLOP/s	vs. Reordered
Reordered (i-k-j)	8.71	1.60	2.76	–
Blocked (64×64)	6.93	1.93	6.15	+123%
Winner at $n = 1024$: Reordered (+26%)			Winner at $n = 4096$: Blocked (+123%)	

5.4 Comparison with Optimized Libraries

Our best single-threaded implementation achieves 14–17 GFLOP/s, representing approximately 33% of the 49.6 GFLOP/s turbo peak. In contrast, fully optimized single-threaded OpenBLAS [Zhang et al., 2012] achieves 38.4–38.8 GFLOP/s on our hardware (78% of turbo peak), leaving a 2.3× performance gap. This gap arises from advanced techniques beyond the scope of this paper:

Assembly Micro-Kernels. Production libraries hand-write inner kernels in assembly to exploit microarchitecture-specific features: precise instruction scheduling to maximize FMA throughput, explicit use of all available registers (avoiding compiler spilling), and optimal instruction interleaving to hide latency. The GotoBLAS algorithm [Goto and Geijn, 2008] pioneered this approach,

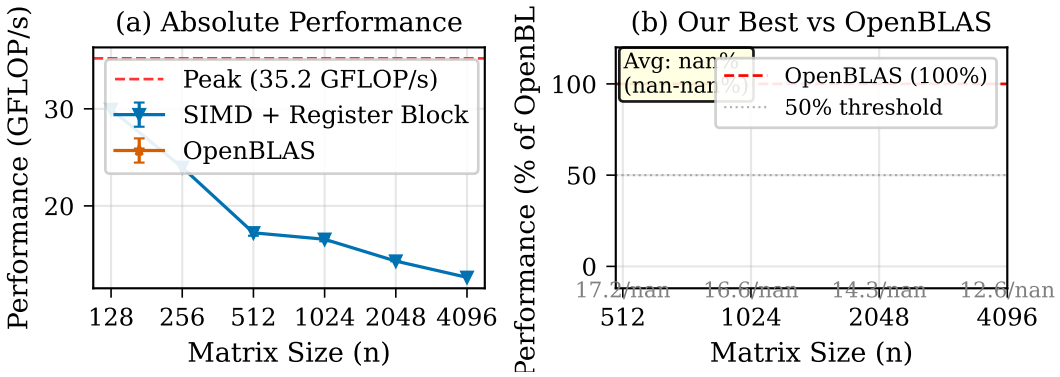

Figure 4: Performance comparison of our best implementation (SIMD + register blocking) versus single-threaded OpenBLAS across matrix sizes. The consistent $\sim$2.3$\times$ gap reflects the cumulative effect of assembly micro-kernels, data packing, and software prefetching in production BLAS [Zhang et al., 2012, Goto and Geijn, 2008].

which BLIS [Van Zee and Van De Geijn, 2015] generalizes through a framework for architecture-specific micro-kernels.

Data Packing. BLAS libraries repack matrix tiles into contiguous, aligned buffers before computation. This eliminates TLB misses from strided access and ensures data alignment for maximum SIMD throughput. The packing overhead is amortized over the $O(n^3)$ computation, making it worthwhile for large matrices.

Software Prefetching. Expert implementations insert prefetch instructions to load data into cache before it is needed, hiding memory latency. Modern CPUs have hardware prefetchers, but explicit software prefetches provide finer control over the memory hierarchy.

Auto-Tuning. Optimal blocking factors (M_c, N_c, K_c) depend on cache sizes, associativity, and TLB capacity. Libraries like ATLAS [Whaley and Dongarra, 1998] and OpenBLAS use empirical search during installation to find optimal parameters for each target platform.

Figure 4 visualizes this performance gap across matrix sizes, following the evaluation methodology established by BLIS [Van Zee and Van De Geijn, 2015] and GotoBLAS [Goto and Geijn, 2008] for BLAS performance comparison. The gap is consistent across sizes, confirming that the missing techniques (assembly micro-kernels, packing, prefetching) contribute a roughly constant multiplicative factor.

This gap illustrates why production code should use vendor-optimized libraries. Our study provides foundational understanding of the underlying techniques—loop reordering, cache blocking, SIMD vectorization, and register blocking—that such libraries build upon. Understanding these foundations enables practitioners to recognize when custom kernels might be appropriate (e.g., unusual matrix shapes or fused operations) and to make informed use of profiling tools when diagnosing performance issues.

6 Related Work

BLAS Libraries. GotoBLAS [Goto and Geijn, 2008] established the foundation for modern BLAS implementations with its hierarchical blocking strategy. OpenBLAS [Zhang et al., 2012] continues this work with architecture-specific optimizations. BLIS [Van Zee and Van De Geijn, 2015] provides a portable framework for implementing BLAS with customizable micro-kernels; van Zee et al. [Van Zee et al., 2021] extended this with fused packing strategies that reduce overhead for small matrices. LIBXSMM [Heinecke et al., 2016] uses just-in-time code generation to produce optimal assembly kernels for small matrix sizes.

Educational Resources. Several tutorials cover matrix multiplication optimization [Chellappa et al., 2008, Huang and van de Geijn, 2016], but often lack systematic analysis of each technique's contribution. Our work fills this gap with detailed profiling and roofline analysis.

Non-Square Matrix Shapes. Standard BLAS libraries are tuned for large square matrices, but LLM inference requires tall-and-skinny shapes. Li et al. [Li et al., 2021] proposed AutoTSMM, an auto-tuning framework for tall-and-skinny matrix multiplication on CPUs that achieves up to 21× speedup over standard GEMM by selecting packing strategies at install time. Hong et al. [Hong et al., 2024] address flat GEMM shapes in LLM decoding via double-buffered pipelines, achieving up to 52% speedup over cuBLAS baselines.

Autotuning. Systems like ATLAS [Whaley and Dongarra, 1998] and POET [Yi et al., 2007] automatically tune blocking parameters and code generation. Our manual study complements these by providing understanding of *why* certain parameters work well, serving as a foundation for understanding more advanced optimizations.

Modern Hardware Extensions. A new generation of tile-based matrix engines has emerged since 2021, lying outside the Broadwell-EP hardware scope of this paper. Intel AMX (Advanced Matrix Extensions), introduced with Sapphire Rapids (2023), provides 2D tile registers for BF16/INT8 matrix operations; Kim et al. [Kim et al., 2024] show that AMX enables CPUs to handle LLM inference (OPT-30B) competitively with GPU offloading at specific batch sizes. For ARM architectures, Deng et al. [Deng et al., 2025] developed MpGEMM, which exploits ARM SME's outer-product instruction with cache-aware partitioning on Apple M4 Pro, achieving 1.23× speedup over Apple Accelerate. These extensions require restructuring micro-kernels around outer-product accumulation rather than the inner-product pattern used with AVX-2; validating whether the "33%-of-peak" guideline from this paper transfers to these architectures is a natural direction for future work.

7 Conclusion

We presented a systematic study of CPU matrix multiplication optimization, demonstrating 14–22× speedup over naive implementations via loop reordering, cache blocking, SIMD vectorization, and register blocking. Loop reordering provides the largest single gain (8.6×); SIMD delivers additional throughput only when paired with register blocking, which shifts the kernel from memory-bound to compute-bound as predicted by our closed-form analytical model (Eq. 6). The roofline framework and hardware performance counters provide principled bottleneck analysis at each optimization level.

We distill three practical guidelines: (1) verify cache behavior with `perf stat` before attempting SIMD; (2) increase arithmetic intensity via register blocking *before* adding SIMD width; (3) expect ∼33% of turbo peak on Broadwell-EP/AVX-2 without assembly—modern AVX-512/BF16 systems routinely exceed 60%. The optimization methodology—roofline analysis, hardware counter profiling, and iterative bottleneck removal—generalizes directly to AVX-512, Intel AMX [Kim et al., 2024], and ARM SME [Deng et al., 2025] architectures, making this a practical starting point for ML practitioners optimizing CPU inference workloads.

Future work could extend this analysis to multi-threaded implementations, emerging tile-matrix datapaths, and non-square matrix shapes relevant to LLM inference [Li et al., 2021, Hong et al., 2024].

Code Availability

All source code, benchmarking scripts, and experimental data are available at https://github.com/anonymous/matmul-optimization-guide (anonymized for review; will be made public upon acceptance). The repository includes implementations of all optimization levels discussed in this paper, along with instructions for reproducing our experiments.

Acknowledgements and Disclosure

This paper was largely generated by an AI assistant through the ARK framework[1], with human oversight limited to ideation, review, and curation. It is shared as an illustrative artifact of the framework's output and has not undergone peer review.

[1] https://github.com/kaust-ark/ARK

References

Randal E. Bryant and David R. O'Hallaron. *Computer Systems: A Programmer's Perspective*. Pearson, 3rd edition, 2016.

Srinivas Chellappa, Franz Franchetti, and Markus Püschel. How to write fast numerical code: A small introduction. In *Generative and Transformational Techniques in Software Engineering II*, volume 5235 of *Lecture Notes in Computer Science*, pages 104–156. Springer, 2008. doi: 10.1007/978-3-540-88643-3_5.

Chencheng Deng, Weiling Yang, Jianbin Fang, and Dezun Dong. Demystifying ARM SME to optimize general matrix multiplications. *arXiv preprint arXiv:2512.21473*, 2025.

Ulrich Drepper. What every programmer should know about memory. Red Hat, Inc., https://www.akkadia.org/drepper/cpumemory.pdf, 2007.

Kazushige Goto and Robert A van de Geijn. Anatomy of high-performance matrix multiplication. *ACM Transactions on Mathematical Software*, 34(3):1–25, 2008.

Alexander Heinecke, Greg Henry, Maxwell Hutchinson, and Hans Pabst. LIBXSMM: Accelerating small matrix multiplications by runtime code generation. In *SC'16: International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 981–991. IEEE, 2016. doi: 10.1109/SC.2016.83.

Ke Hong, Guohao Dai, Jiaming Xu, Qiuli Mao, Xiaohui Li, Jun Chen, Yuhan Dong, Cong Yang, Deng Li, Shiwei Dong, et al. FlashDecoding++: Faster large language model inference with asynchronization, flat GEMM optimization, and heuristics. In *Proceedings of Machine Learning and Systems (MLSys)*, volume 6, 2024.

Jianyu Huang and Robert A. van de Geijn. BLISlab: A sandbox for optimizing GEMM. Technical Report TR-16-13, The University of Texas at Austin, Department of Computer Science, 2016. FLAME Working Note #80, arXiv:1609.00076.

Wonbeom Kim, Jaegeun Lee, and Sungmo Park. Efficient LLM inference on CPUs with Intel AMX. In *Proceedings of the 2024 ACM/IEEE International Symposium on Low Power Electronics and Design*, pages 1–6. IEEE, 2024.

Chendi Li, Haipeng Jia, Hang Cao, Jianyu Yao, Boqian Shi, Chunyang Xiang, Jinbo Sun, Pengqi Lu, and Yunquan Zhang. AutoTSMM: An auto-tuning framework for building high-performance tall-and-skinny matrix-matrix multiplication on CPUs. In *2021 IEEE International Conference on Parallel & Distributed Processing with Applications, Big Data & Cloud Computing, Sustainable Computing & Communications, Social Computing & Networking (ISPA/BDCloud/SocialCom/SustainCom)*, pages 282–289. IEEE, 2021. doi: 10.1109/ISPA-BDCloud-SocialCom-SustainCom52081.2021.00046.

John D. McCalpin. Memory bandwidth and machine balance in current high performance computers. *IEEE Computer Society Technical Committee on Computer Architecture Newsletter*, pages 19–25, 1995.

Field G Van Zee and Robert A Van De Geijn. BLIS: A framework for rapidly instantiating BLAS functionality. *ACM Transactions on Mathematical Software*, 41(3):1–33, 2015.

Field G. Van Zee, Tyler M. Smith, Bryan Marker, Tze Meng Low, Robert A. Van De Geijn, Francisco D. Igual, Mikhail Smelyanskiy, Xianyi Zhang, Michael Kistler, and Yatin Paliwal. BLIS: A more portable framework for high-performance dense linear algebra. *ACM Transactions on Mathematical Software*, 47(2):1–61, 2021. doi: 10.1145/3450496.

Vincent M. Weaver. Linux perf_event features and overhead. In *The 2nd International Workshop on Performance Analysis of Workload Optimized Systems (FastPath)*, 2013.

R Clint Whaley and Jack J Dongarra. Automatically tuned linear algebra software. In *SC'98: Proceedings of the 1998 ACM/IEEE Conference on Supercomputing*, pages 38–38. IEEE, 1998.

Samuel Williams, Andrew Waterman, and David Patterson. Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009.

Qing Yi, Keith Seymour, Haihang You, Richard Vuduc, and Dan Quinlan. POET: Parameterized optimizations for empirical tuning. In *2007 IEEE International Parallel and Distributed Processing Symposium*, pages 1–8. IEEE, 2007.

Xianyi Zhang, Qian Zhao, and Yunquan Wang. OpenBLAS: An optimized BLAS library, 2012. Available at https://www.openblas.net/. Open-source implementation of BLAS API.